

ML-Based Adaptive Learning System for Neurodiverse Students - Project Report

1. Project Overview

Project Title

ML-Based Adaptive Learning System for Neurodiverse Students

One-Line Description

A full-stack adaptive learning web application that predicts a student's learning style and provides personalized study recommendations, behavioral insights, analytics, and parent/teacher reports.

Purpose / Problem It Solves

The project helps students, parents, and teachers understand how a learner engages with educational content. It collects learning behavior, questionnaire answers, subject preferences, and short support-check task results, then predicts whether the student is best supported through Visual, Audio, or Text-based learning resources.

For neurodiverse students, a single teaching format may not work well for everyone. This system aims to make learning more personalized by identifying preferred learning patterns and suggesting practical support strategies.

Target Users / Audience

- Neurodiverse students who need personalized learning support.
- Teachers who want insight into student learning preferences and engagement.
- Parents who need understandable reports about strengths, risks, and recommended support.
- Schools or learning centers exploring adaptive learning tools.

Project Type

Full-stack web application with a machine learning backend.

The project includes:

- React frontend.
- Flask API backend.
- Scikit-learn machine learning models.
- CSV-based dataset storage.
- Analysis scripts for model evaluation and visualization.

2. Tech Stack & Tools

Languages Used

- JavaScript / JSX for the frontend.
- Python for backend, machine learning, preprocessing, and analysis.
- CSS for styling.
- CSV for dataset storage.

Frameworks and Libraries

Frontend:

- React ^19.1.0
- React DOM ^19.1.0
- Vite ^7.0.0
- @vitejs/plugin-react ^5.0.0
- Chart.js ^4.5.0
- Firebase ^12.11.0

Backend:

- Flask
- Flask-CORS
- NumPy
- Pandas
- Scikit-learn
- Joblib

Machine Learning:

- RandomForestClassifier
- LogisticRegression
- SVC
- KNeighborsClassifier
- LabelEncoder
- StandardScaler
- SimpleImputer
- train_test_split
- cross_val_score
- accuracy_score
- classification_report
- confusion_matrix

Analysis / Visualization:

- Matplotlib
- Seaborn
- Power BI dataset preparation script

Databases / Storage

- CSV datasets in the dataset/ folder.
- Trained ML model files stored as .pkl files using Joblib.
- Firebase Authentication and Firestore are configured on the frontend, if the required Vite environment variables are provided.
- The Flask endpoints for behavior tracking and report saving currently acknowledge received data but do not persist it to a database.

External APIs or Services

- Firebase Authentication.
- Firebase Firestore.
- Browser Web Speech APIs for speech recognition and speech synthesis.
- External educational/tool links in the recommendation toolkit, such as YouTube, Google Docs, Quizlet, Notion, Excalidraw, and text-to-speech tools.
- NASA-hosted educational videos used in behavioral verification lessons.

Dev Tools

- npm as the frontend package manager.

- Vite as the frontend development/build tool.
- pip for Python dependency installation.
- Joblib for model serialization.
- Flask development server for local API execution.

3. Project Architecture

High-Level Architecture

The system follows a client-server architecture with an ML pipeline:

1. The React frontend collects student profile details, questionnaire responses, behavioral signals, timed-task results, and subject preferences.
2. The frontend sends prediction input to the Flask backend through the /predict endpoint.
3. The Flask backend loads a trained machine learning model from ml/learning_style_model.pkl.
4. The model predicts the student's learning style.
5. The frontend displays the result, charts, recommendations, adaptive support messages, analytics, and a parent/teacher report.
6. Separate Python scripts handle preprocessing, training, model comparison, and analysis visualizations.

Folder / Module Structure

```
project-folder/
|-- README
|-- PROJECT_REPORT.md
|-- vite.config.js
|-- frontend/
|   |-- package.json
|   |-- package-lock.json
|   |-- vite.config.js
|   |-- src/
|       |-- App.jsx
|       |-- main.jsx
|       |-- styles.css
|       |-- firebase.js
|       |-- useAdaptiveEngine.js
|       |-- assets/
|           |-- adaptlearn-logo.svg
|           |-- cloud wallpaper images
|-- backend/
|   |-- api.py
|   |-- requirements.txt
|   |-- test_api.py
|-- ml/
|   |-- preprocess.py
|   |-- train_model.py
|   |-- random_forest_model.py
|   |-- logistic_model.py
|   |-- svm_model.py
|   |-- knn_model.py
|   |-- predict.py
|   |-- test_model.py
|   |-- check_features.py
|   |-- add_synthetic_features.py
|   |-- learning_style_model.pkl
|   |-- logistic_model.pkl
|   |-- svm_model.pkl
|   |-- knn_model.pkl
```

```

| |-- feature_names.pkl
|-- analysis/
| |-- feature_importance.py
| |-- cross_validation.py
| |-- confusion_matrix.py
| |-- confidence_analysis.py
| |-- plot_graph.py
| |-- log_analysis.py
| |-- powerbi_dataset_dashboard.py
| |-- analysis_graph/
|-- dataset/
| |-- combined_final_dataset.csv
| |-- dataset_with_synthetic_features.csv
| |-- train_data.csv
| |-- test_data.csv

```

How Components Interact

- frontend/src/main.jsx mounts the React application.
- frontend/src/App.jsx contains the main UI, state management, assessment flow, behavior tracking, prediction call, charts, recommendations, and report display.
- frontend/src/firebase.js initializes Firebase only when the required VITE_FIREBASE_* environment variables are available.
- backend/api.py exposes REST endpoints and loads the trained model.
- ml/train_model.py trains the main Random Forest model and saves it as learning_style_model.pkl.
- dataset/combined_final_dataset.csv provides the main training and analytics data.
- analysis/*.py scripts generate model evaluation charts and summary visuals.

Design Patterns Used

- Client-server architecture.
- Single Page Application pattern on the frontend.
- REST API pattern for backend communication.
- Machine learning pipeline pattern: preprocessing -> training -> evaluation -> serialized model -> inference API.
- Component-based UI design in React.
- Rule-based adaptive support logic inside the frontend, combined with ML-based learning-style prediction.

4. Features & Functionality

Complete Feature List

- Home/introduction screen.
- Student login form using student name and ID.
- Optional Firebase account creation and login.
- Voice-assisted input using browser speech recognition.
- Text-to-speech page guidance.
- Behavioral verification tasks using learning media and typed responses.
- Multi-task support check with:
 - timed typing task,
 - drag-and-drop ordering task,
 - memory recall task,
 - audio recall task,
 - short comprehension task.
- Learning habit questionnaire with sliders.

- Subject preference collection.
- ML-based prediction of learning style.
- Fallback client-side prediction if backend prediction fails.
- Dashboard showing predicted learning style and confidence.
- Chart.js visualizations.
- Dataset summary analytics from the backend.
- Personalized recommendations for Visual, Audio, and Text learners.
- Direct learning toolkit links based on predicted style.
- Parent/teacher report with strengths, risks, and support suggestions.
- Adaptive support engine that reacts to behavior indicators such as long pauses, repeated corrections, and task difficulty.
- Backend endpoints for login, prediction, behavior tracking, report submission, and dataset summary.
- ML training scripts for Random Forest, Logistic Regression, SVM, and KNN.
- Analysis scripts for feature importance, cross-validation, confidence analysis, and confusion matrix.

Core Workflow

1. The student opens the React app.
2. The student enters name and student ID.
3. The app optionally supports authentication through Firebase.
4. The student completes behavior verification tasks.
5. The student completes the timed multi-task support check.
6. The student answers learning habit questions using sliders or voice input.
7. The student selects subject preferences.
8. The frontend builds a feature payload from the answers.
9. The frontend sends the payload to POST /predict.
10. Flask loads the trained model and predicts a learning style.
11. The frontend displays the predicted style, confidence value, charts, recommendations, toolkit links, analytics, and report.
12. Behavior and report data are sent to backend acknowledgement endpoints.

User Flows

Student flow:

1. Home -> Login -> Create Account or continue.
2. Behavioral Check -> Timed Support Check.
3. Learning Habit Assessment -> Subject Preferences.
4. Dashboard -> Recommendations -> Analytics -> Report.

Teacher/parent flow:

1. Review predicted style.
2. Review student input indicators.
3. Review support-check interpretation.
4. Read recommended interventions.
5. Use the report for follow-up planning.

5. Setup & Installation

Prerequisites

- Python 3.9 or later recommended.
- Node.js and npm.

- Modern browser with JavaScript enabled.
- Optional: Firebase project credentials if authentication and Firestore are required.

Backend Installation

From the project root:

```
cd backend
pip install -r requirements.txt
```

Run the Flask API:

```
python api.py
```

The backend runs locally at:

```
http://127.0.0.1:5000
```

Frontend Installation

From the project root:

```
cd frontend
npm install
npm run dev
```

The Vite development server will show the local frontend URL, commonly:

```
http://localhost:5173
```

Environment Variables / Config

Firebase is optional, but if enabled, create a frontend .env file with:

```
VITE_FIREBASE_API_KEY=
VITE_FIREBASE_AUTH_DOMAIN=
VITE_FIREBASE_PROJECT_ID=
VITE_FIREBASE_STORAGE_BUCKET=
VITE_FIREBASE_MESSAGING_SENDER_ID=
VITE_FIREBASE_APP_ID=
VITE_FIREBASE_MEASUREMENT_ID=
```

If these values are missing, firebaseReady becomes false and Firebase services are not initialized.

ML Training Setup

Install backend/ML dependencies first, then run:

```
python ml/train_model.py
```

This trains the main Random Forest model and saves:

```
ml/learning_style_model.pkl
ml/feature_names.pkl
```

6. Code Walkthrough

Key Files and Roles

frontend/src/App.jsx

- Main application file.
- Manages page flow, student state, assessment answers, behavior data, timed tasks, voice input, prediction calls, charts, recommendations, analytics, and reports.
- Contains helper functions such as mockPrediction, calculateTextAccuracy, scoreKeywordRecall, buildFeatureRows, buildBehaviorInsights, buildSupportCheckInsights, buildAdaptiveEngine, and buildReportSummary.

frontend/src/firebase.js

- Reads Firebase configuration from Vite environment variables.
- Initializes Firebase Auth and Firestore only when required config values exist.

frontend/src/styles.css

- Contains styling for the full frontend interface.

backend/api.py

- Main Flask server.
- Loads the trained model from ml/learning_style_model.pkl.
- Defines the REST API endpoints.
- Reads dataset/combined_final_dataset.csv for dataset summary analytics.

ml/train_model.py

- Loads the combined dataset.
- Handles missing values.
- One-hot encodes categorical columns.
- Trains a Random Forest classifier.
- Saves the trained model and feature list.

ml/preprocess.py

- Fills missing values.
- Encodes categorical features.
- Scales feature values.
- Splits the data into train and test CSV files.

ml/logistic_model.py, ml/svm_model.py, ml/knn_model.py

- Train alternative models for comparison.
- Save model artifacts for Logistic Regression, SVM, and KNN.

analysis/feature_importance.py

- Trains a Random Forest model and plots feature importance.

analysis/cross_validation.py

- Runs 5-fold cross-validation on a Random Forest classifier.

analysis/confusion_matrix.py

- Builds a confusion matrix and classification report.

analysis/confidence_analysis.py

- Uses predicted class probabilities to analyze confidence.

Important Functions / Logic

mockPrediction(answers)

- Frontend fallback prediction.
- Compares video, audio, and text activity values.
- Returns Visual, Audio, or Text with a basic confidence score.

calculateTextAccuracy(target, response)

- Compares typed response characters against the target sentence.
- Used in timed typing support checks.

scoreKeywordRecall(response, keywords)

- Counts how many expected keywords appear in a student's response.
- Used in audio recall and memory-style scoring.

buildBehaviorInsights(behaviorData)

- Converts interaction data into readable behavioral insights.
- Looks at typing level, pauses, corrections, and navigation patterns.

buildSupportCheckInsights(...)

- Combines timed typing, ordering, memory, audio recall, and comprehension task scores.
- Produces support messages and diagnostic notes.

buildAdaptiveEngine(...)

- Creates adaptive interventions based on observed behavior and support-check results.
- Produces student-facing and teacher-facing messages.

analyzeResults()

- Builds the prediction payload.
- Calls POST <http://127.0.0.1:5000/predict>.
- Uses backend prediction when available.
- Falls back to mockPrediction() if the backend is unavailable.

predict() in backend/api.py

- Receives JSON input.
- Extracts model features in the expected order.
- Converts input into a NumPy array.
- Calls model.predict().
- Returns the predicted learning style and the features used.

Complex Logic / Algorithms

- Random Forest classification is the primary ML algorithm.
- Alternative classifiers include Logistic Regression, SVM, and KNN.
- Missing numeric values are filled with mean values.
- Missing categorical values are filled with mode or Unknown.
- Categorical features are encoded using one-hot encoding or label encoding depending on the script.
- Frontend adaptive support combines rule-based behavior analysis with ML prediction.

7. Data Flow

Input -> Processing -> Output

Frontend input:

- Student name and ID.
- Account details.
- Questionnaire answers.
- Subject preferences.
- Slider values.
- Voice input.
- Timed task responses.
- Behavior signals such as keystrokes, pauses, backspaces, page visits, media plays, and

tool clicks.

Processing:

- Frontend builds a prediction payload.
- Backend extracts expected features.
- Trained ML model predicts learning style.
- Frontend combines prediction with behavioral support insights.
- Charts and reports are generated in the frontend.

Output:

- Learning style prediction.
- Confidence score shown in the frontend.
- Personalized learning recommendations.
- Adaptive support messages.
- Analytics charts.
- Parent/teacher report.

Dataset Structure

Main dataset file:

```
dataset/combined_final_dataset.csv
```

Columns include:

```
hand_raise  
resource_visits  
announcement_views  
discussion_posts  
free_time  
study_time  
absences  
final_grade  
learning_style  
total_actions  
video_actions  
audio_actions  
text_actions  
platform
```

Target column:

```
learning_style
```

Expected prediction features in the Flask API:

```
hand_raise  
resource_visits  
announcement_views  
discussion_posts  
free_time  
study_time  
absences  
final_grade  
total_actions  
video_actions  
audio_actions  
text_actions  
platform_Unknown
```

API Endpoints

GET /

Health check endpoint.

Response:

```
Learning Style Prediction API Running
```

POST /student-login

Receives basic student login data.

Request:

```
{
  "student_name": "Student Name",
  "student_id": "STU001"
}
```

Success response:

```
{
  "message": "Student login received",
  "student": {
    "student_name": "Student Name",
    "student_id": "STU001"
  }
}
```

Validation error:

```
{
  "error": "student_name and student_id are required"
}
```

POST /predict

Predicts the student's learning style.

Request:

```
{
  "hand_raise": 50,
  "resource_visits": 70,
  "announcement_views": 40,
  "discussion_posts": 30,
  "free_time": 5,
  "study_time": 3,
  "absences": 2,
  "final_grade": 75,
  "total_actions": 180,
  "video_actions": 80,
  "audio_actions": 40,
  "text_actions": 60,
  "platform_Unknown": 0
}
```

Response:

```
{
  "prediction": "Visual",
  "features_used": {
    "hand_raise": 50,
    "resource_visits": 70,
    "announcement_views": 40,
    "discussion_posts": 30,

```

```
    "free_time": 5,  
    "study_time": 3,  
    "absences": 2,  
    "final_grade": 75,  
    "total_actions": 180,  
    "video_actions": 80,  
    "audio_actions": 40,  
    "text_actions": 60,  
    "platform_Unknown": 0  
  }  
}
```

POST /behavior-track

Receives behavior tracking data.

Request:

```
{  
  "student_id": "STU001",  
  "student_name": "Student Name",  
  "behavior_data": {},  
  "support_check": {},  
  "adaptive_engine": {}  
}
```

Response:

```
{  
  "message": "Behavior tracking received"  
}
```

POST /save-report

Receives generated report data.

Request:

```
{  
  "student_id": "STU001",  
  "student_name": "Student Name",  
  "report_summary": {},  
  "behavior_insights": {},  
  "support_check": {},  
  "adaptive_engine": {},  
  "student_snapshot": {}  
}
```

Response:

```
{  
  "message": "Report received"  
}
```

GET /dataset-summary

Returns aggregate dataset analytics.

Response:

```
{  
  "total_students": 100,  
  "learning_style_distribution": {  
    "Visual": 40,  
    "Audio": 30,  
    "Text": 30  
  }  
}
```

```

    },
    "average_metrics": {
      "hand_raise": 45.5,
      "resource_visits": 62.1
    },
    "performance_bands": {
      "high_grade": 30,
      "medium_grade": 50,
      "low_grade": 20
    }
  }
}

```

8. Challenges & Solutions

Challenge 1: Supporting Different Learning Needs

The project needs to support students who may respond better to visual, audio, or text-based learning.

Solution:

The system collects multiple indicators and predicts a learning style. It also provides separate recommendation maps for Visual, Audio, and Text learners.

Challenge 2: Combining ML Prediction With Behavioral Support

Learning style alone does not explain all student needs.

Solution:

The frontend adds behavior tracking and timed support-check tasks. Rule-based insight functions convert behavior patterns into practical support suggestions.

Challenge 3: Backend Availability

The frontend depends on the Flask API for prediction.

Solution:

The frontend includes `mockPrediction()` as a fallback so the app can still show a result if the backend is unavailable.

Challenge 4: Handling Missing Dataset Values

The dataset contains missing values in some columns.

Solution:

Training scripts fill numeric columns with mean values and categorical columns with mode or Unknown.

Challenge 5: Feature Alignment Between Training and Inference

ML models require prediction-time features to match training-time features.

Solution:

The backend defines a fixed `FEATURES` list and reads incoming request values in that exact order. The training script also saves `feature_names.pkl` for reference.

Challenge 6: Making Reports Understandable

ML outputs can be difficult for non-technical users.

Solution:

The frontend translates raw features and scores into plain-language interpretations for parents and teachers.

9. Testing

Test Files / Testing Approach

The project includes:

```
backend/test_api.py
ml/test_model.py
analysis/cross_validation.py
analysis/confusion_matrix.py
analysis/confidence_analysis.py
```

backend/test_api.py sends a POST request to the prediction endpoint and prints the response.

analysis/cross_validation.py performs 5-fold cross-validation for model evaluation.

analysis/confusion_matrix.py creates a confusion matrix and classification report.

analysis/confidence_analysis.py checks model probability output for prediction confidence.

How to Run Backend API Test

First start the backend:

```
python backend/api.py
```

Then run:

```
python backend/test_api.py
```

Note: The current backend/test_api.py payload uses older behavioral feature names such as typing_speed_wpm and keystroke_delay, while the current backend expects the 13 features listed in FEATURES. Updating the test payload to match the backend feature list would make the test more accurate.

How to Run ML Evaluation

```
python analysis/cross_validation.py
python analysis/confusion_matrix.py
python analysis/confidence_analysis.py
```

How to Run Frontend Build Check

```
cd frontend
npm run build
```

10. Future Improvements

Current Limitations

- The backend /save-report and /behavior-track endpoints do not persist data yet.
- Firebase is configured, but the app depends on environment variables being supplied.
- The prediction endpoint does not currently return true model probability/confidence.
- The frontend uses fallback confidence when the API does not provide confidence.
- frontend/src/useAdaptiveEngine.js is empty, while adaptive logic currently lives inside App.jsx.
- Some ML scripts use slightly different preprocessing approaches, which may create

inconsistent model behavior.

- backend/test_api.py does not match the current /predict feature schema.
- No formal unit test suite is configured for the React frontend.

Suggested Enhancements

- Store behavior logs and reports in Firestore or a backend database.
- Return predict_proba() confidence values from the Flask prediction endpoint.
- Move adaptive support logic into useAdaptiveEngine.js as a reusable hook.
- Add automated tests with pytest for Flask endpoints.
- Add frontend tests using a React testing framework.
- Standardize one preprocessing pipeline for training and inference.
- Add model versioning and metadata.
- Add authentication checks before saving reports.
- Add teacher/admin dashboard for reviewing multiple students.
- Improve accessibility with keyboard navigation audits and screen-reader labels.
- Add export options for reports, such as PDF or printable HTML.

11. Summary

This project is an adaptive learning system that helps students discover the study format that may suit them best. It asks questions, observes learning behavior, runs short support tasks, and uses a machine learning model to predict whether the student may benefit most from visual, audio, or text-based learning. The app then turns that result into practical recommendations, charts, and a parent/teacher report, making the output useful for both students and the adults supporting them.